Computação Móvel
Mobile Computing
CM

Tutorial 1 - Hello World

Pedro Fazenda
pedrofazenda@enautica.pt

Carlos Gonçalves
carlosgoncalves@enautica.pt

Abstract

This assignment familiarizes students with Kotlin and Android development. It starts with installing IntelliJ IDEA and working through Basic Types and Control Flow concepts. Students then set up Android Studio and build their first "Hello, World!" Android application. The assignment also introduces Classes and Objects to strengthen object-oriented programming skills. In addition, students are introduced to agentic programming, giving them the opportunity to explore intelligent, autonomous coding approaches and experiment with other mobile development platforms beyond Android. Optional challenges are provided for those who want to explore further. Overall, the goal is to establish a strong Kotlin foundation while keeping the learning process engaging and enjoyable.

Deadline: March 15th, 2026

Hello Kotlin. Hello Mobile Android World!

February 27, 2026

Contents

1 Introduction **1**
1.1 Official documentation . . . 1
1.2 AI Usage Policy and Academic Integrity . . . 1
1.3 Assignments Submission and Deadline Policy . . . 2
1.4 Reports [**AC YES, AI YES**] . . . 2
1.5 Document structure . . . 4

2 Getting Started with Kotlin [**AC NO, AI NO**] **4**
2.1 Exercise 1 . . . 5
2.2 Exercise 2 . . . 5
2.3 Exercise 3 . . . 6

3 Android Introduction and Preliminary Steps **6**
3.1 Overview of the Android Development Environment . . . 6
3.2 Download and installation of Android Studio . . . 7
3.3 Installation of Android development libraries (SDKs) . . . 8
3.4 Checking for Updates . . . 9
3.5 Creating Android Virtual Devices (VDs/AVDs) . . . 9
3.6 Setting Up Your Phone for App Development . . . 12

4 Building Your First Android App [**AC YES, AI NO**] **14**
4.1 Create a “Hello Mobile World” Application . . . 14
4.2 Improve your application - Hello World v2 [**AC YES, AI NO**] . . . 18

5 About Android Studio **19**
5.1 Useful shortcuts . . . 19
5.2 Logging and Monitoring App State [**AC YES, AI NO**] . . . 20
5.3 Building a System Info App [**AC YES, AI NO**] . . . 22

6 Exploring Kotlin Again [**AC NO, AI NO**] **22**
6.1 Base requirements . . . 23
6.2 Extended requirements . . . 24
6.3 Expected Implementation . . . 25
6.4 Expected Console Output . . . 26
6.5 Assessment Criteria . . . 26

7 Return and Enhance your Android experience [**10%**] [**AC YES, AI NO**] **27**

8 Assisted code generation - MIP [**20%**] [**AC YES, AI YES**] **27**

1 Introduction

This assignment introduces students to the fundamentals of Kotlin and Android development. The purpose of this guide is to provide a comprehensive introduction to the language, development environment, and basic programming concepts essential for the development of mobile applications. At the end of this assignment, students will acquire a basic understanding of the development of Kotlin and Android.

1.1 Official documentation

For official documentation, in Kotlin and Android Development, see:

Kotlin docs: `https://kotlinlang.org/docs/home.html`; and

Android Developers Guide: `https://developer.android.com/guide`.

1.2 AI Usage Policy and Academic Integrity

The emergence of artificial intelligence in code generation represents a major breakthrough for the software industry. It offers a significant advantage in software development by accelerating code writing, improving productivity, helping to debug, and lowering barriers to entry for novice programmers. AI-assisted coding tools can also support learning by providing examples, explanations, and alternative solutions, exposing students to a wider range of programming practices and design patterns.

However, these benefits are accompanied by important cognitive drawbacks. Excessive reliance on AI tools can hinder the development of fundamental problem-solving skills, reduce the depth of understanding of programming concepts, and encourage superficial learning. Students may become less capable of designing solutions independently, debugging unfamiliar code, or critically evaluating the correctness and efficiency of generated programs. Over time, this dependency can negatively affect analytical thinking, creativity, and long-term retention of knowledge. For these reasons, the use of AI-assisted coding must be carefully balanced with activities that promote independent reasoning and the mastery of core programming skills.

Throughout lectures and assignments in this course, students will be given tasks for which the use of AI-driven tools is entirely prohibited. This includes auto-completion functionalities, which are explicitly forbidden in certain activities. These restrictions are established for pedagogical purposes.

Each task will be clearly labeled with **AC** (Auto-complete) and/or **AI** (Artificial Intelligence assistance), followed by **YES** or **NO**, indicating whether the use of these features is permitted or not.

To turn off Auto-complete by AI Assistant go to Settings (on Windows/Linux Settings, on MAC: IntelliJ IDEA → Settings), then go to Tools → AI Assistant, then disable options: *Inline code completion* and *Automatic suggestions* and deactivate any other AI code completions. Only the standard non-AI code completion is allowed. Artificial

Intelligence assistance refers to any method of generating code from written, spoken, or otherwise described instructions (e.g., text or voice prompts).

The objective is to ensure that students are able to solve specific problem sets independently, starting from a blank slate, without relying on external assistance. Developing these foundational skills is essential to build a strong understanding of the underlying concepts and acquire the desired competence.

There will also be sections of the coursework in which the use of AI tools is permitted or even mandatory, reflecting their growing relevance in modern software development practices. Clear instructions will be provided for each assignment to indicate when AI assistance is allowed or required.

When using AI or auto-complete tools, you remain fully responsible for all content and must be able to justify every part of it. Failure to do so will result in those parts being excluded from grading.

1.3 Assignments Submission and Deadline Policy

The following requirements are valid for all assignments (TP1 ... TP4) and for the final project.

All development must be conducted under **version control** using **GitHub**. The latest commit pushed before the deadline indicated on the assignment cover page will be considered for grading. The commit history must clearly reflect the progression of your work throughout the project, with entries spanning from the beginning to the end (not only a final commit). Compliance with this requirement (as with all requirements) is part of the grading criteria.

Late submissions will be subject to a penalty. For each day after the deadline, 1 point (out of 20) will be deducted from the assignment grade, calculated up to the time of the final commit (including the report Markdown file(s)).

All assignments must be presented to the instructor and assessed during class hours within the two weeks following the submission deadline. **Attendance is compulsory; failure to meet this requirement will result in a grade of 0 for the respective assignment.**

1.4 Reports [AC YES, AI YES]

For each GitHub repository submitted as part of an assignment, a report must also be included in the repository in Markdown format.

See Markdown Cheatsheet or Quickstart for Writing on GitHub for guidance on this markup language.

You must include a `README.md` file. If additional Markdown files are needed, place them in the `docs` directory (e.g., `docs/report_2_1.md`).

You may use AI tools to help you write the reports.

Follows the required structure for the `README.md` file:

```
# Assignment X     Title
<!-- Replace X and Title -->
```

```
Course: ___________________
Student(s): ________________
Date: _____________________
Repository URL: ___________

---

## 1. Introduction
<!-- Purpose of the assignment, problem description, and objectives. -->

## 2. System Overview
<!-- High-level description of the solution, main features, and use cases.
   -->

## 3. Architecture and Design
<!-- Architecture, folder structure, design patterns, and justification of
   key decisions. -->

## 4. Implementation
<!-- Implementation details: main modules, components, algorithms, and
   relevant code excerpts. -->

## 5. Testing and Validation
<!-- Testing strategy, test cases, scenarios, edge cases, and known
   limitations. -->

## 6. Usage Instructions
<!-- How to run the project: requirements, setup, configuration, and
   execution steps. -->

---

# Autonomous Software Engineering Sections - only for [AC OK, AI OK]
   sections

## 7. Prompting Strategy
<!-- Describe the prompts used with AI tools, their purpose, and how they
   evolved. Include representative examples. -->

## 8. Autonomous Agent Workflow
<!-- Explain how AI tools or agents contributed to development: planning,
   coding, debugging, testing, documentation, etc. -->

## 9. Verification of AI-Generated Artifacts
<!-- Describe how you verified correctness of AI-generated code/designs
   (testing, manual review, static analysis, etc.). -->

## 10. Human vs AI Contribution
<!-- Clearly state which parts were primarily human-developed and which
   were AI-assisted. -->

## 11. Ethical and Responsible Use
<!-- Reflect on risks, limitations, biases, or inappropriate outputs from
   AI tools and how they were handled. -->

---

# Development Process
```

```
## 12. Version Control and Commit History
<!-- Describe how version control was used. The commit history must
   reflect continuous work (not only final commits). -->

## 13. Difficulties and Lessons Learned
<!-- Main challenges, mistakes, insights, and skills acquired during the
   assignment. -->

## 14. Future Improvements
<!-- Possible extensions, optimizations, or features that could be added
   in future work. -->

---

## 15. AI Usage Disclosure (Mandatory)
<!-- List all AI tools used (e.g., ChatGPT, Copilot, etc.), how they were
   used, and confirmation that you remain responsible for all content. -->
```

1.5 Document structure

The tutorial begins with an introduction to Kotlin, covering its Basic Types and Control Flow to help students understand fundamental programming concepts. Once students have a basic grasp of the language, they will proceed with the development of their first mobile Android application, starting with a simple "Hello, World!" project.

Following this, the assignment introduces the concepts of Classes and Objects, reinforcing object-oriented programming principles within Kotlin.

Finally, students return to Android programming to deepen their experience and explore AI-assisted code generation.

These topics provide a foundation for building simple Android applications.

2 Getting Started with Kotlin [AC NO, AI NO]

Let us learn the core concepts of Kotlin and how to get started with it. Start by downloading and installing the latest version of IntelliJ IDEA. Then create your first Kotlin project with a simple Hello, World program. Once you have successfully executed your code, you can move on to building the applications outlined below.

Keep in mind that this task is meant to be an enjoyable and exciting challenge. Resist the temptation to use any AI-based code assistance tools. Throughout your career, you will have plenty of opportunities to take advantage of these tools. At this stage, your focus should be on developing your programming skills, critical thinking, debugging abilities, and mental flexibility. Everything you need to complete these exercises can be found in the Kotlin documentation:

Kotlin Docs - `https://kotlinlang.org/docs/home.html`.

At this stage, focus on the following sections under Concepts:

1. Basic Types
2. Control Flow

3. Classes and Objects

For the Kotlin exercises in this assignment, you must create a Kotlin project using Maven and a Git repository. For each Android app in the assignment, you must create a separate Android project with its own repository. For other assignments, follow the same approach.

Create a Kotlin project in IntelliJ named `CM_TP1` (use similar names for other assignments).

In IntelliJ, select: `New → Project → Kotlin, Maven, Create Git repository`.

Run the `Main.kt` file to ensure everything is properly configured.

Create the packages required for this assignment: `cm`, `cm.exer_1`, `cm.exer_2`, `cm.exer_3`, `cm.virtual_library`.

Add the file `cm.exer_1/exer_1.kt` and copy the content of existing `Main.kt` file to it (save it, add it to repository - and for all new files) and run it to verify the setup again. Open Commit tab (below Project icon tab, in top leftmost bar, or look for it), select all files, add the Commit message: initialize project structure with Kotlin and Maven configuration and do `Commit`.

For this exercise it is only required that you use a local repository and do `Commit`. Later we will move to remote repository, and then we will do `Commit and Push`.

Then delete its contents and start implementing the exercise.

Repeat the actions for each remaining exercise as you reach it.

Enjoy the process and have fun with the code!

2.1 Exercise 1

Create and initialize an integer array with the first 50 perfect squares $(1^2, 2^2, 3^2, ..., 50^2)$ dynamically, using the following:

- a) Using `IntArray` constructor;
- b) Using a range and `map()`;
- c) Using `Array` with constructor;

2.2 Exercise 2

Create a console-based calculator that allows users to perform various operations. Your implementation should incorporate the following Kotlin features:

1. The **when** expression for handling different operations.
2. **Exceptions** to handle potential errors (e.g., division by zero, invalid input).
3. **String templates** and **string formatting** for a clear and structured output.

Your calculator should support the following:

1. Basic arithmetic operations: Addition, subtraction, multiplication, and division.

2. Boolean operators: AND (`&&`), OR (`||`), NOT (`!`).
3. Bitwise shift operators: Left shift (`shl`), right shift (`shr`).
4. Show the results in decimal, hexadecimal, and boolean.

Note: Ensure that the program provides user-friendly prompts and gracefully handles invalid input.

2.3 Exercise 3

A ball is dropped from a height of 100 meters. Each time it bounces, it reaches 60% of its previous height. Write a program using `generateSequence` to model the bounce heights, filter to keep only bounces that reach at least 1 meter high, take the first 15 qualifying bounces, convert to a list, and print the heights rounded to 2 decimal places. Print the sequence.

Hint: You will need to think about how to represent each bounce height based on the previous one. Also, refer to the official documentation at Kotlin Sequences - generateSequence [1] and analyze the following code to understand its functionality.

3 Android Introduction and Preliminary Steps

First, an introduction to the Android development environment is presented to provide a high-level overview of what Android is and how it works.

Second, to enable the development and execution of Android applications, this tutorial covers the installation of the IDE, the development libraries (Software Development Kit — SDK), and the emulator. It also ensures that the latest software versions are installed and configures your Android phone as a development target.

3.1 Overview of the Android Development Environment

Android OS was initially developed by Android Inc., a company co-founded by Andy Rubin in the early 21st century with the goal of creating a powerful and innovative mobile operating system. In 2005, Google acquired Android Inc., recognizing its potential to transform the mobile industry. Following the acquisition, Andy Rubin was appointed as the director of Google's mobile platforms, leading the evolution of Android into what would become the world's most widely used operating system for smartphones and tablets.

Android OS is built on the **Linux Kernel**, making Android devices essentially Linux-based computers. The development and maintenance of the Android system are managed by the **Android Open Source Project (AOSP)**, an initiative led by Google to promote an open and flexible mobile ecosystem.

The execution of Android applications is handled by **Android Runtime (ART)**, which is specifically designed for mobile environments and uses ahead-of-time (AOT) and just-in-time (JIT) compilation to optimize performance.

[1] `https://kotlinlang.org/api/core/kotlin-stdlib/kotlin.sequences/generate-sequence.html`

When Android code is compiled, it produces Dalvik Executable (**.dex**) files.

These **.dex** files, together with resources and manifest files, are packaged into an **.apk** file — the standard format for installing Android applications on devices. For distribution through Google Play, applications are typically published as Android App Bundles (**.aab**), which are later converted into optimized APKs for each device.

There are four main components (plus one, Layouts) in Android applications [2]:

- **Activities** - (organizational layer) an Activity supports a screen for user interaction;
- **Layout Containers** (presentation layer) a Layout is a container for user interface elements in a screen;
- **Services** - (background processing layer) a Service is a component that runs in background, providing services for Activities;
- **Broadcast Receivers** (notification layer) a Broadcast Receiver is a component that allows the Activities to receive notifications from the system (and react to them);
- **Content Providers** (storage layer) a Content Provider is a component that stores and provides data for Activities.

There are also some important helper classes:

- Android **Fragments** - they are reusable parts of the user interface;
- Android **Intents** - they enable communication between components.

These elements will be introduced in more detail later. For now, we focus on Android **Activities** and **Layouts**.

An **Activity** is a screen that provides context for user interactions. Applications may have multiple activities to manage content on mobile screens with various sizes and resolutions. Each activity includes a **Layout Container** that holds UI elements and can change dynamically.

A **Layout Container** organizes UI elements, called **Widgets**, that include text, graphics, 3D content, video, menus, and animations. Layouts are based on the **ViewGroup** class, while Widgets derive from the **View** class.

Android graphics, such as images and animations, are called **Drawables**. Event handling allows UI elements to respond to user actions and other events.

3.2 Download and installation of Android Studio

Android Studio is the official Android Integrated Development Environment (IDE) for Android Development.

[2] `https://developer.android.com/guide/components/fundamentals`

1. Download the latest Android Studio installation from the following link:

   `https://developer.android.com/studio` (Current version: Panda 1 — 2025.3.1)

   The Android Studio package includes all the tools necessary for the development of android applications:

   - **Android Studio IDE** - to edit, run, debug Android code
   - **Android SDK and Tools Manager** - to install Android SDKs and tools
   - **Android Emulator** - to run Android apps in Android emulated devices
   - **Android Virtual Device (AVDs) manager** to create and manage AVDs.

   Note: If you were programming in Java, you would also need to install the Java Development Kit (JDK) and the Java Runtime Environment (JRE).

2. Execute the "android-studio-panda1-windows.exe" file obtained with the previous download to install the Android Studio package (if you are using windows, otherwise choose your appropriate installation).

3. Run the Android Studio IDE.

4. You will go through several configuration steps to install Android Studio.

3.3 Installation of Android development libraries (SDKs)

There are several versions of the Android platform.

They have a version number, a codename, and an API level, such as **Android 12 Snow Cone (API 31)**, **Android 13 Tiramisu (API 33)**, and **Android 14 Upside Down Cake (API 34)**.

To develop code for a specific platform, you need the corresponding libraries, known as the **Software Development Kit (SDK)**.

For example, to develop code for **Android 13 (API 33)**, you need its Android SDK platform and tools.

The management of installed SDKs, as well as the installation of new ones, is handled through the **SDK Manager**, which is available in **Android Studio**.

It is recommended to install the latest stable version of Android, such as **Android 14 (API 34)**, along with at least one older version, like **Android 10 (API 29)** or another that matches the highest version supported by the student's device.

1. Open the Android SDK Manager. Do it in the "Welcome to Android Studio" window (when there is no Project opened), by selecting the **"More Actions (3 dots on top right)"** drop down list and choosing **SDK Manager** (see Figure 1) or when we are working on a project by opening the menu **Tools** > **SDK Manager** or by selecting the **SDK Manager icon** in the toolbar.

2. In the SKD Manager panel (see Figure 2) select the Android version you wish to develop that is not installed by checking them and then press the "OK" button. Select the "Show Package Details" checkbox to see in detail the components that are part of each version of the Android API.

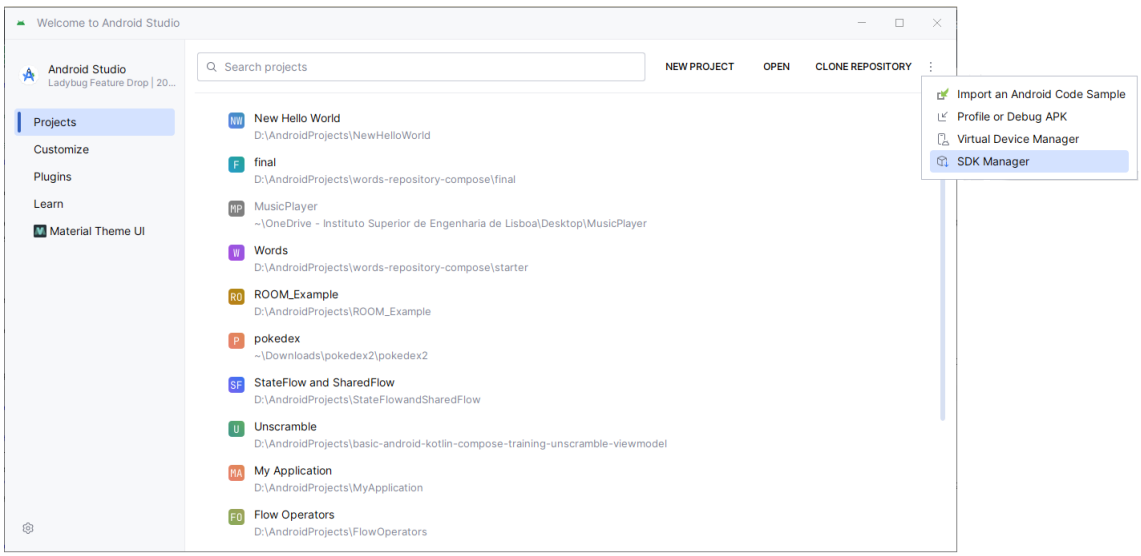

Figure 1: Configure in Welcome window.

3.4 Checking for Updates

It is recommended to keep **Android Studio** and all platforms and tools up to date. After installation, it is a good time to check for the latest updates. In the **Android Studio Welcome screen**, when no project is opened, updates can be checked by selecting the **Settings (gear icon)** → **Check for Updates** option (see Figure 3).

When working on a project, updates can be checked via the menu **Help** → **Check for Updates...** on Windows/Linux, or **Android Studio** → **Check for Updates...** on macOS.

Note: It is advisable to install only **stable versions** and avoid Beta versions unless specifically required.

3.5 Creating Android Virtual Devices (VDs/AVDs)

An **Android Virtual Device (AVD)** is a software-based emulator that replicates the behavior of a physical Android device.

AVDs allow testing Android applications on different Android versions and hardware configurations without requiring a physical device.

The **AVD Manager** is a tool that facilitates the creation of AVDs for various device types, including **Phone, Tablet, TV, Automotive, and Wear OS**.

It provides several predefined hardware profiles, but users can also create or import custom profiles as needed.

It is recommended to have at least two AVDs:

- One **Pixel 9 Pro** smartphone
- One **Pixel Tablet**

The **AVD Manager** allows creating AVDs based on a specific hardware profile and an Android platform version.

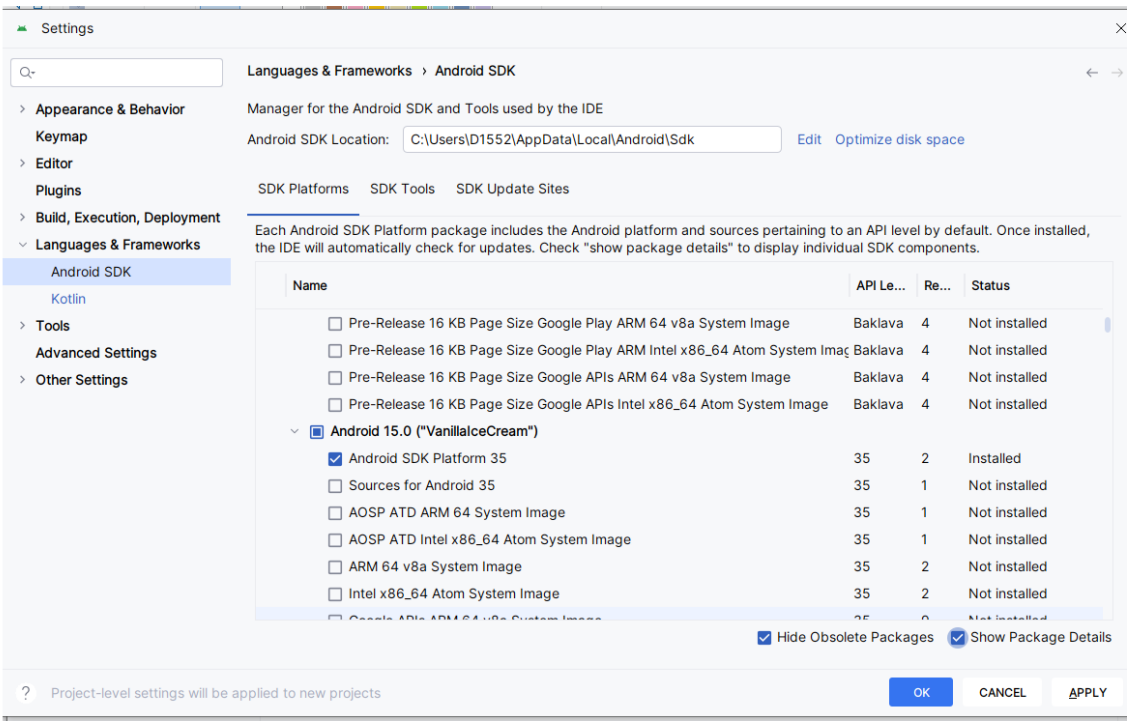

Figure 2: SDK Manager: SDK Platforms.

To set up an AVD, you must select both a hardware profile and an Android system image.

It is advisable to choose the **latest stable Android API version** compatible with your physical device. This ensures that applications run consistently on both the virtual device

1. Open the **AVD Manager** in one of the following ways:
   - From the **Welcome to Android Studio** window (when no project is opened), select the **"More Actions"** drop-down menu, and choose **Device Manager**.
   - When working on a project, open the menu **Tools** → **Device Manager**.
   - Click on the **Device Manager icon** on the toolbar.
2. Create two **virtual devices (VDs)**: one for a **Pixel 9 Pro** smartphone and another for a **Pixel Tablet**.
   - Start by clicking the **"Create Device"** button in the **Virtual Device Configuration** panel (Select Hardware); see Figure 4.
   - Choose the appropriate hardware profile for the selected virtual device.
   - Click **"Next"** to proceed.
3. In the VD system image panel, see Figure 5, select the Android platform version, hardware architecture (x86, ...) and the supported Google services (Google Play, etc.), download it if necessary and then click the "Next" button. Use the recommended configuration and leave the remaining parameters, for now, set to "default" and finish.

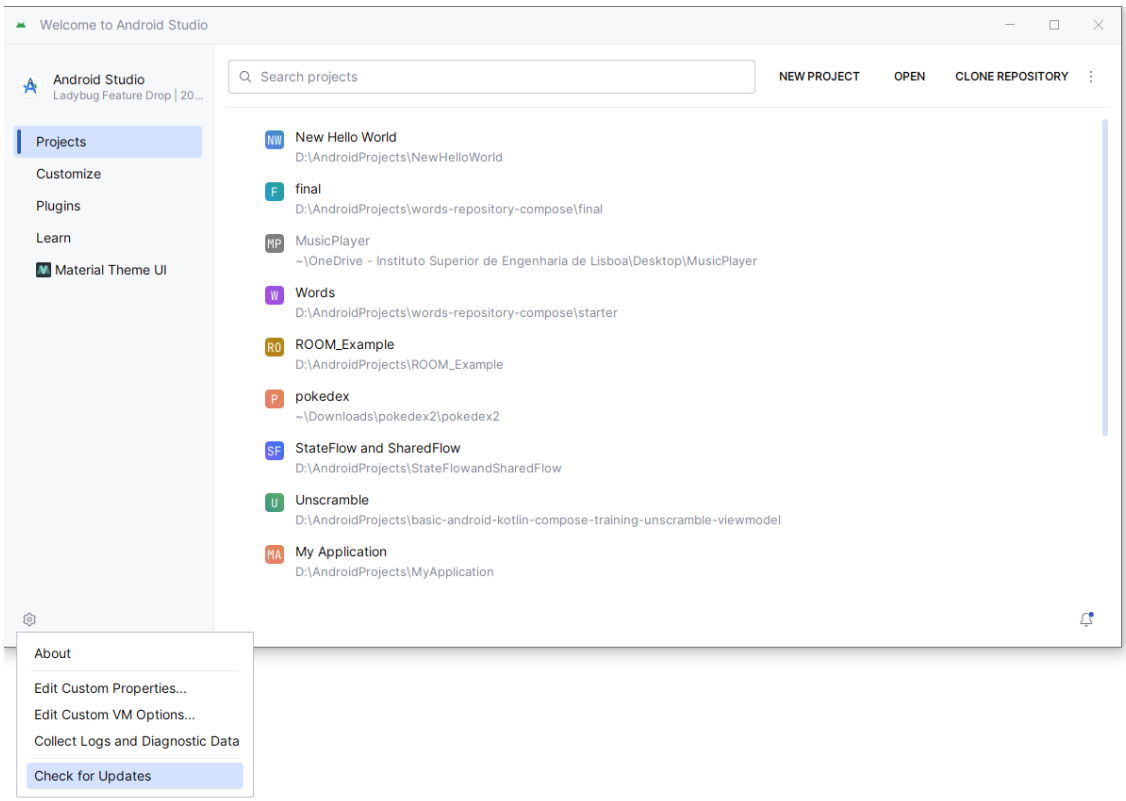

Figure 3: Android Studio Welcome: "Check for Updates".

4. Start Pixel 9 Pro AVD. After creating the emulators, you can see them in the Device Manager panel (see Figure 6). In that list, click the "play" button on the **Pixel 9 Pro** smartphone emulator to launch it.

   Note 1: The first launch of a **virtual device (VD)** may take **several minutes**, depending on the **computing power** of the PC. Interrupting the VD during the startup process may cause **corruption**, requiring its reconfiguration or deletion.

   Note 2: Once the **virtual device (VD)** is started, it should **remain running** throughout development. Instead of stopping or closing it, simply **re-deploy** the application to the VD after making code changes.

   Note 3: When creating an emulator, you can choose between the **"Snapshot"** and **"Host GPU"** options. If you choose the **"Snapshot"** option, the emulator saves a copy of its memory state when stopped, similar to hibernation. This allows for a much **faster startup** the next time it is launched. If you choose the **"Host GPU"** option, the emulator **renders graphics more efficiently** by utilizing the PC's graphics card, improving performance.

   Note 4: When selecting the **Android API version** during emulator creation, you can choose a **VD image** based on either an **ARM** or **Intel** CPU architecture. A VD based on **Intel architecture** runs significantly **faster** on an **Intel-based PC**, as it eliminates the need to translate **ARM instructions** into the PC's Intel architecture.

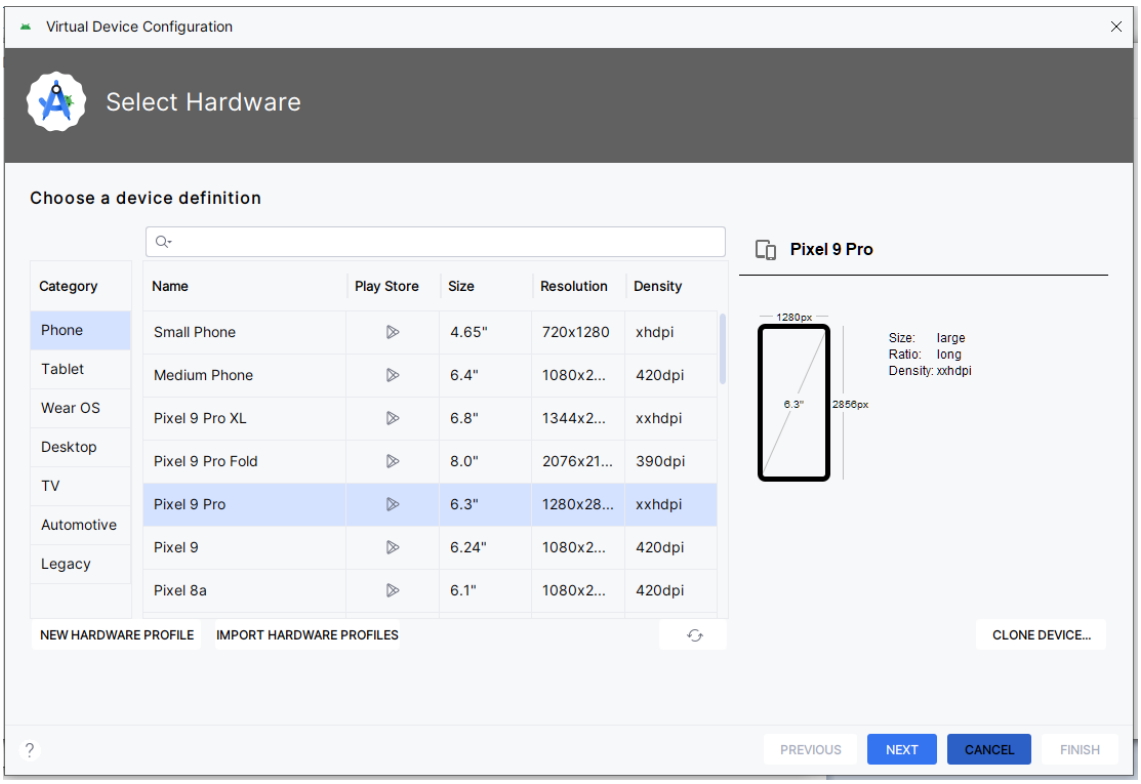

Figure 4: Virtual Device: choosing system image.

3.6 Setting Up Your Phone for App Development

This setting described in this section may change depending on your phone. In general, for most phones, you can use the procedures described below. If these do not work on your phone, search online for the specific steps for your device.

1. **Enable Installation from Unknown Sources**

   With **Android 8.0 (Oreo)**, the process of allowing installation from unknown sources has changed.

   - **For Android 8.0 and above:**
     - Go to **Settings** > **Apps & notifications** > **Special app access** > **Install unknown apps**.
     - Select the app (e.g., your browser or file manager) you intend to use to install APKs.
     - Enable the option **"Allow from this source"**.
   - **For Android 7.1 and below:**
     - Go to **Settings** > **Security**.
     - Enable the option **"Install unknown apps"**.
     - Confirm by tapping **"OK"**.

   Note: The exact path may vary slightly depending on your device manufacturer and the Android version.

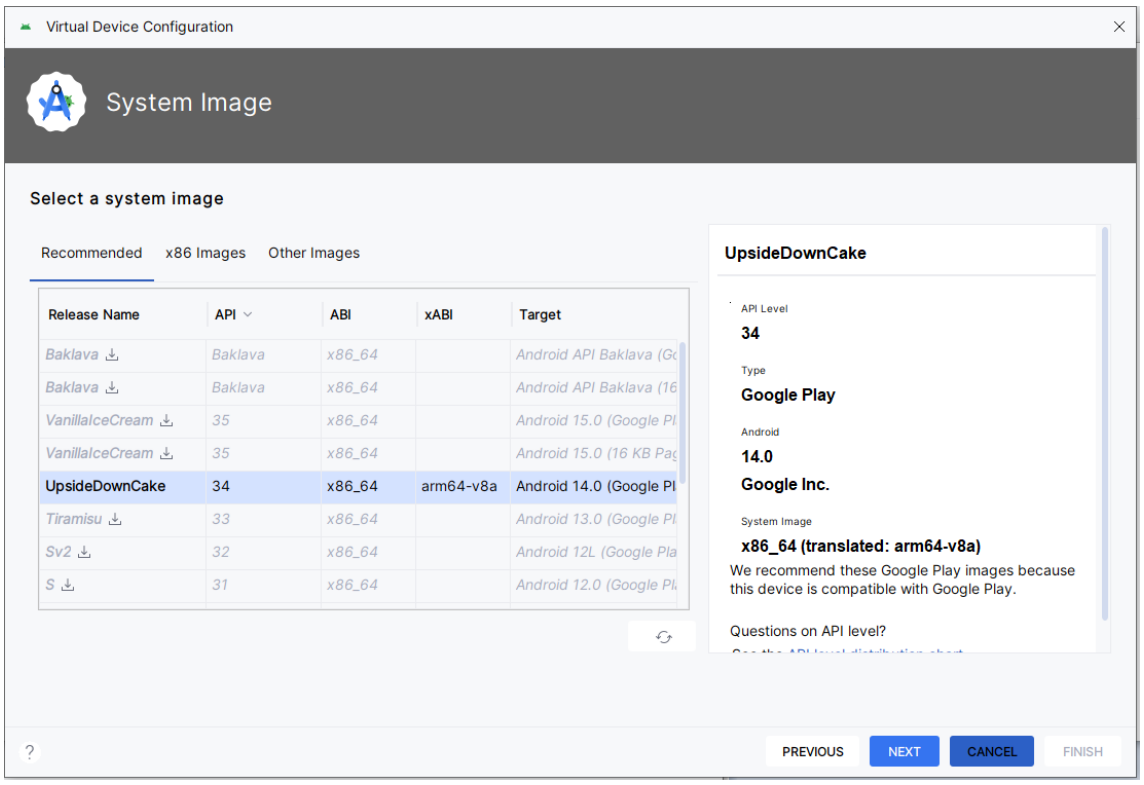

Figure 5: Virtual Device: choosing system image.

Figure 6: Device Manager: list of existing Virtual Devices.

2. **Enable Developer Options**

   To install and debug applications on your phone, you must activate **Developer options**.

   - Open **Settings** > **About the phone**.
   - Tap **Build number** seven times.
   - Enter your device's PIN or password if asked.
   - Return to **Settings**, and you will find **Developer options** listed.

   Note: The location of the **Build number** can vary; it is usually found in **About phone** or **Software information**.

3. **Enable USB Debugging**

   After activating Developer options, enable USB debugging:

   - Open **Settings** > **Developer options**.

- Enable the option **"USB debugging"**.

Note: USB debugging allows your computer to communicate with your device for development purposes. Ensure that you trust the computer to which you are connecting.

For more detailed information, refer to the official Android documentation:

`https://developer.android.com/studio/command-line/adb`

Note: Always exercise caution when enabling these settings, as they can expose your device to security risks.

4 Building Your First Android App [AC YES, AI NO]

4.1 Create a "Hello Mobile World" Application

1. **Create a New Android Project** Open **Android Studio** and navigate to **File** > **New** > **New Project** to open the project creation dialog (see Figure 7).

   - Select the **Empty Views Activity** option.
   - Click **Next** to proceed.

   This option will generate a basic application that contains a single activity, providing a simple starting point for development.

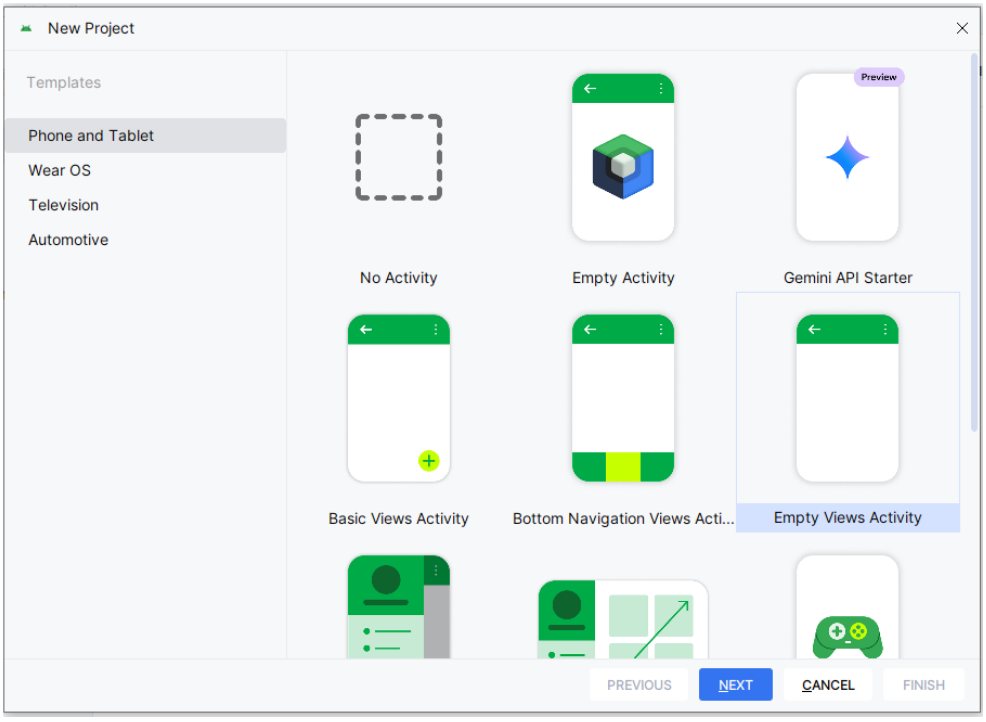

Figure 7: Choose your project - Empty Views Activity

2. **Configure Project Details**

   - Enter the application name: **Hello World**.

- Specify the package name: `cm_aXXXXX.helloworld` (see Figure 8).
- **Replace** `aXXXXX` with your student number.
- **Use this base package name for all your projects.**
- Set the project save path as: `cm-path\code\HelloWorld`, where `cm-path` is the directory for CM.
- **Select Kotlin as the programming language**.
- Choose **API 24: Android 7.0 (Nougat)** as the Minimum API level.
- Click **Finish** to generate the project.

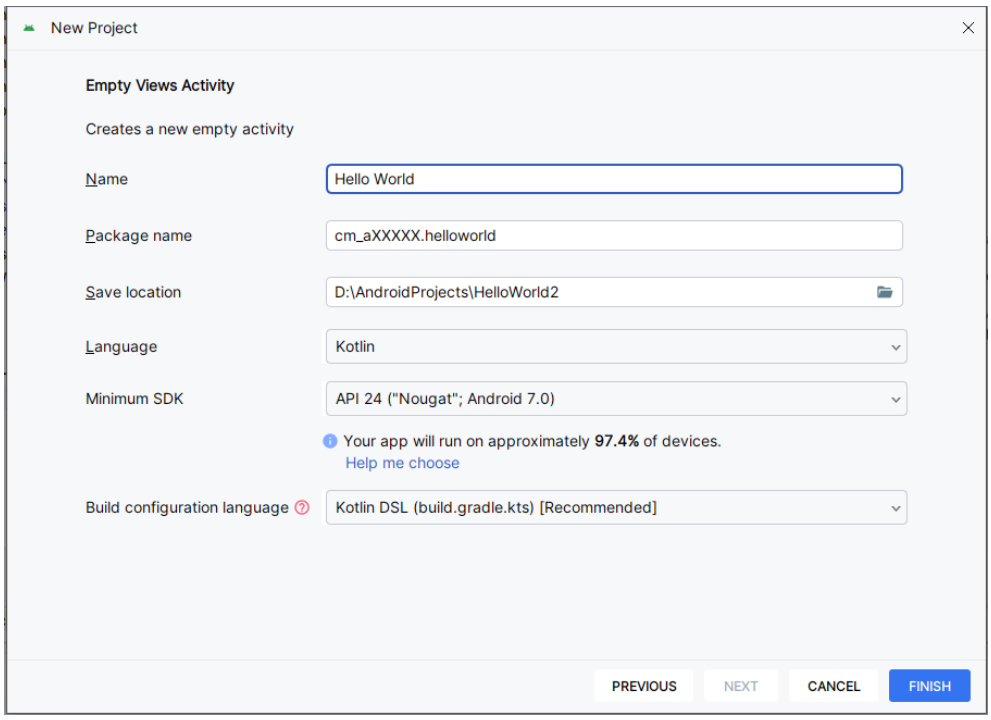

Figure 8: Configure you project.

Note 5: When selecting the **Minimum API level**, the configuration wizard will display the estimated percentage of devices that support each version, along with the new features introduced in that version. Take the time to review the new features of each API version before making a selection.

Note 6: After pressing the **Finish** button, some installations of the IDE may report an error related to Gradle JDK:

No matching variant of com.android.tools.build:gradle:...

If you encounter this issue, change the **Gradle JDK** to another version in the following location: **File** > **Settings** > **Build, Execution, Deployment** > **Build Tools** > **Gradle**. Download a different JDK version if necessary.

3. Inspect the generated code. As you can see in Figure 9 the Android Studio creates three sections for files: **manifests**, **java**, and **res**. An application must contain a **manifest** file that defines essential app information, including:

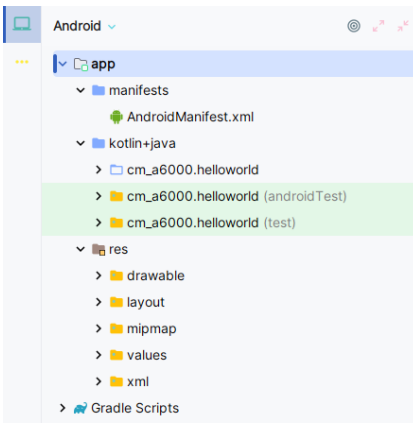

Figure 9: Main files.

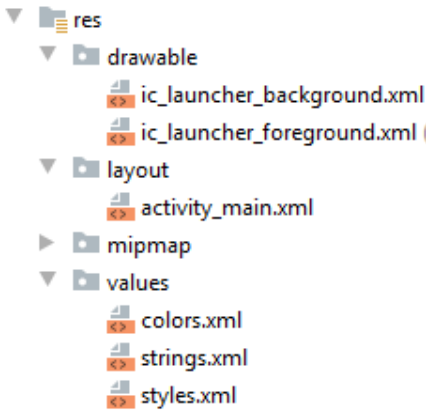

Figure 10: Resource files.

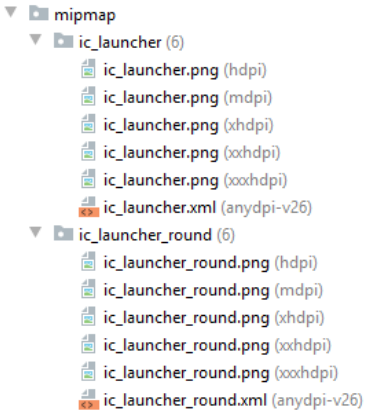

Figure 11: Res. img. files.

- The app's package name.
- Declared components such as **activities, services, broadcast receivers, and content providers**, along with their properties.
- Required permissions.
- Hardware and software features needed by the app.

The **java** section contains Java (`.java`) and Kotlin (`.kt`) files. It should include Activity files and all other Java/Kotlin source files. These files are organized according to the defined package structure. By default, the project contains one **Activity**.

The **res** (resources) section, as illustrated in Figure 10, contains the following resource directories:

- **drawable** for image and graphic resources.
- **layout** for XML layout files.
- **mipmap** for launcher icons (see Figure 11).
- **values** for XML resource values such as strings, themes, and dimensions.

Inspect the content of all files.

To explore the **activity_main.xml** file, view it in different modes: **Code**, **Split**, and **Design**, by selecting the corresponding icons located at the top-right side of the editor window.

4. **Run the Application on the Pixel 9 Pro AVD** Execute the application in the **Pixel 9 Pro** AVD using one of the following methods:

   - Select **Run** > **Run "app"** from the menu.
   - Press **Shift + F10**.
   - Click the **green play button** in the toolbar (located to the right of the device drop-down box, next to "No Devices" in Figure 3).

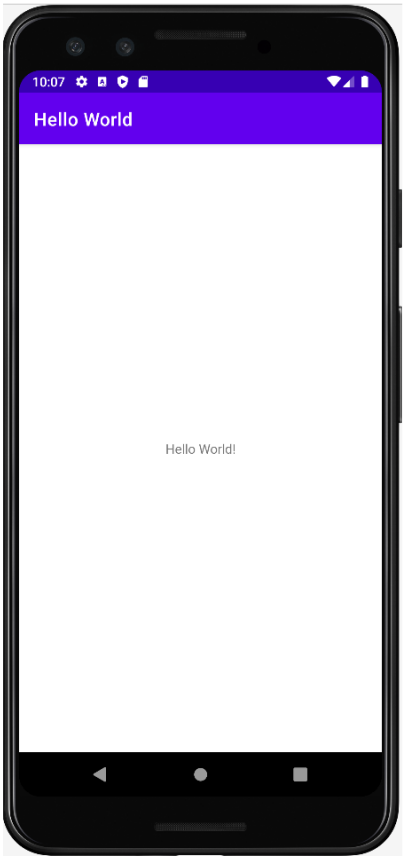

Figure 12: Hello World app.

In the device selection window, choose your **Pixel 9 Pro AVD** smartphone and click the **"OK"** button. The AVD will display the contents shown in Figure 12. Verify the created elements on the emulator screen to ensure the application runs correctly.

Note 7: This process may take some time. Please be patient. If the execution becomes unresponsive, try closing the AVD and restarting the application.

5. **Define Strings in the `strings.xml` File**. Strings should be defined in the `strings.xml` file to support internationalization (this will be covered later).

   In `strings.xml`, each string is identified by a unique name. To reference a string in XML layouts, use `@string/strname` (if the string is named `strname`).

   Modify the TextView text to use `strings.xml`:

   - Open `activity_main.xml` and select the existing string `"Hello World!"`.
   - Press **ALT + ENTER** and choose **"Extract string resource"**.
   - Enter `hello_string` as the resource name and press **OK**.

   Verify that the `android:text` property of the `TextView` now contains:

   `@string/hello_string`,

   and that the new string `hello_string` is declared in `strings.xml`.

   Run the application to apply the changes.

Modify and Test Strings in `strings.xml`

- Change the string `hello_string` (in `strings.xml`) to `"Hello Android World"`.
- Run the application again to see the updated text.

Change the Application Name

- Modify the app name in `strings.xml` to `"Hello World V1"`.
- Run the application again to verify the change.

Run and Debug the Application on a Physical Device. Use **Android Debug Bridge (ADB)** to run and debug the application on a real device:

- Connect your device using a **USB** cable.
- Alternatively, use **WiFi** for debugging.

For more details, refer to the official Android documentation:

`https://developer.android.com/studio/command-line/adb`.

4.2 Improve your application - Hello World v2 [AC YES, AI NO]

1. Change the app name (in strings.xml) to "Hello World V2". Run the application (do the same for remaining points).

2. **Change the TextView** to the top and change some properties. You should have the window in **Split mode** (select in top right corner), with the **Palette view** at the left, the design editor in the middle, and the **Attributes view** at the right. In the design editor, select the TextView and in its bottom constraint give a left mouse click with the CTRL pressed to remove that constraint. To add a new constraint, if necessary, just stretch a new one from a source point (one of the circles) to an anchor point such as, e.g., another TextView, the edge of the screen or a horizontal/vertical barrier. Set its attributes: textColor (use colors placed in colors.xml, e.g., purple_500), textSize and textStyle, to look the same as in 13.

3. **Add a second TextView** with "My First App" text (strings to strings.xml), by dragging one from the palette to the middle of the graphic editor. Connect the top graphical constraint to the bottom of the other TextView. Change the attributes of the TextViews to change: layout_width (set to 0dp to use all width), textSize, textAlignment and background to look the same as in 13.

4. **Add an image** to the screen, as shown in Figure 13. Download a small image from the web and add it to the "drawable" folder. The image name should have only lower-case letters. From the palette view drag an ImageView element to the layout of activity_main.xml. In "Pick a resource view" select the image. It will add to the ImageView: app:srcCompat="@drawable/smileygood" (if smile is the name of the image file). You will see a yellow warning because the image does not have a description. Add a content Description (alternative text) to ImageView: android:contentDescription="Just smile" (strings to strings.xml).

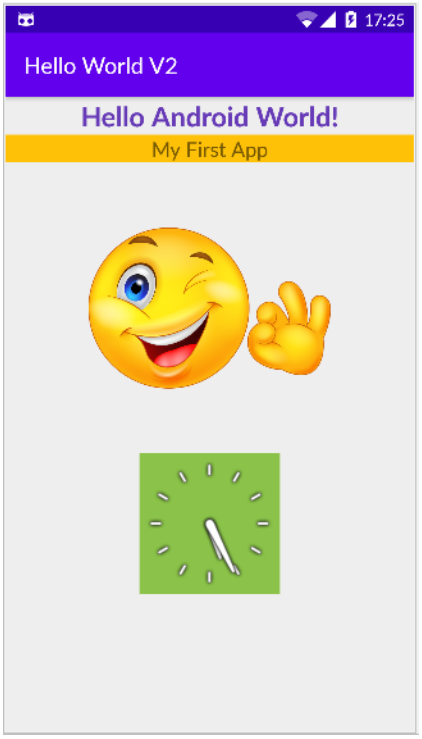

Figure 13: Hello World app v2.

5. **Add a CalendarView** to the layout, as shown in Figure 13.
6. **Add a Landscape layout variant**.
7. **Add an app icon**.

5 About Android Studio

5.1 Useful shortcuts

1. **Rename**. This action allows you to rename an artifact (such as a class, method, variable, or package). Ensure that all occurrences of the artifact are renamed consistently throughout the project. To activate this action:
   - Go to: **Menu** > **Refactor** > **Rename**.
   - Or press **Shift + F6**.
2. **Format Code**. This action automatically formats the code to improve readability and maintain consistent styling. To activate this action:
   - Go to: **Menu** > **Code** > **Reformat Code**.
   - Or press **Ctrl + Alt + L**.
3. **Organize Imports**. This action helps to automatically manage the import statements. It adds necessary imports (if referenced artifacts are correctly used) and removes unused ones. To activate this action:
   - Go to: **Menu** > **Code** > **Optimize Imports**.
   - Or press **Ctrl + Alt + O**.

5.2 Logging and Monitoring App State [AC YES, AI NO]

1. **Observing Messages in Logcat**. The **Logcat** view allows you to visualize the execution messages sent by the device.

   - If the **"No Filters"** option is selected in the right drop-down box, all messages from the device will be displayed.
   - To filter messages related only to a specific application, select the option **"Show only selected application"** in the drop-down box.
   - The second left drop-down box specifies the application being observed.
   - The first drop-down box allows you to select the device, while the third drop-down box sets the message log level.

2. **Sending Messages to Logcat**. To send output to **Logcat**, we can use the standard console output, i.e., `System.out`. To send a message indicating that we are inside the `onCreate` method of the Activity, follow these steps:

   - Open the **MainActivity.kt** file.
   - At the end (but still inside) of the **onCreate** method, insert the following line:

     ```
     println(this@MainActivity.localClassName + " onCreate")
     ```

   - Run the application and observe the **Logcat** output. You should see the message appearing in the log.

   In the next tutorials, we will use the **Log** class to send messages more effectively.

3. **Setting a Dynamic String**. Now, we will store the message in the `strings.xml` file.

   - Double-click on the Activity string to select it.
   - Press **ALT + ENTER** and choose **"Extract String Resource"**.
   - In the **Extract Resource** dialog box, enter the following:
     - **Resource name:** `activity_oncreate_msg`
     - **Resource value:** `"Activity %1$s onCreate"`

   Strings with dynamic content, such as the one in `onCreate`, use the `String.format()` method. In this case, `%1$s` acts as a placeholder for the first argument, treated as a string. Now, complete the call to `getString()` by adding a second argument that specifies the value to be displayed. This should result in:

   ```
   getString(R.string.activity_oncreate_msg,
       this@MainActivity.localClassName)
   ```

   Run the application again to see the updated output. Refer to Figure 14 for the expected results.

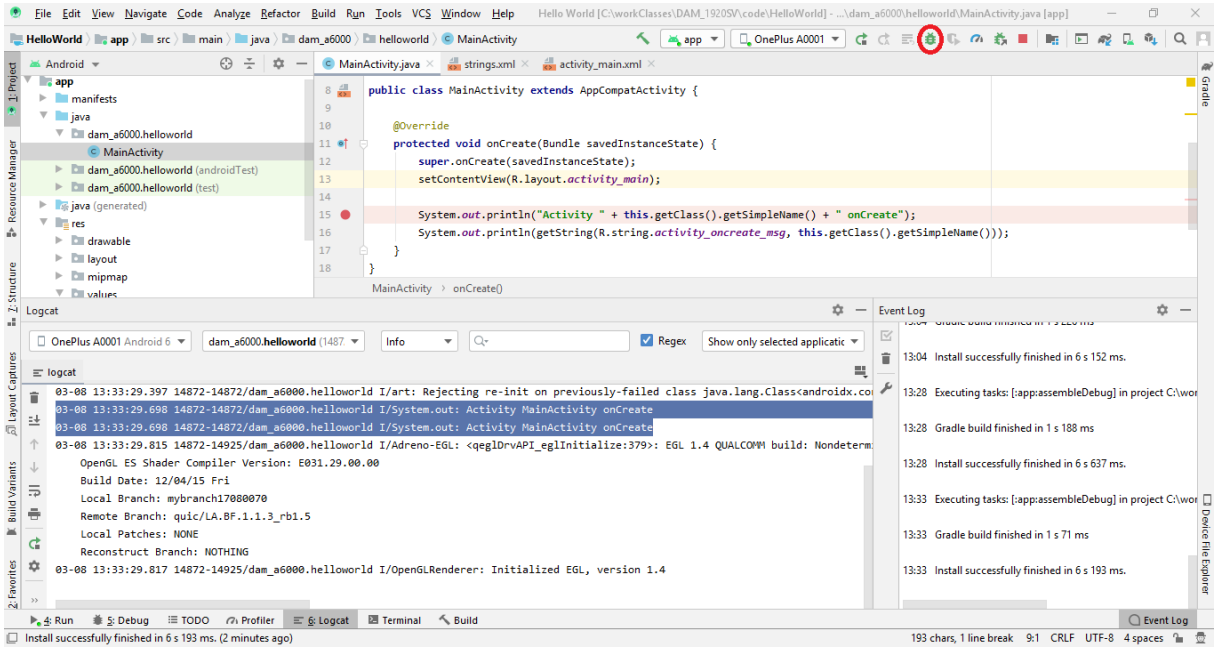

Figure 14: AS with dynamic string and breakpoint.

4. **Capturing the Screen**. The **Logcat** view provides two icons to capture the device screen:

   - **Screen Capture** – represented by a **camera icon** - allows taking a screenshot of the device screen.
   - **Screen Record** – represented by a **green play icon**, enables recording the screen activity.

5. **Execute in Debug Mode** . To run the application in **Debug Mode**, you first need to set a **breakpoint** to pause execution at a specific point.

   - Place a breakpoint on the **print** line from the previous step by clicking in the left margin/column of the code editor.
   - A **red circle** should appear, indicating the break point (see the red circle on line 15 in Figure 14).

   Run the app in Debug Mode using one of the following methods:

   - Go to **Menu** > **Run** > **Debug 'app'**.
   - Press **Shift + F9**.
   - Click the **Bug icon** in the toolbar (highlighted with a red circle in Figure 14).

   Once the application is running in Debug Mode:

   - Observe that **Logcat** does not yet display the `print(...)` message.
   - Execute the **print** instruction by selecting: **Menu** > **Run** > **Step Over**, or pressing **F8**.
   - The **Logcat** should now display the message: `"Activity MainActivity onCreate"`.

To stop a debug session, click the **red square** in the toolbar. Check the **Variables view** inside the **Debug** tab to inspect the values of existing variables.

5.3 Building a System Info App [AC YES, AI NO]

Create a new application that displays information about the current device build on a **MultiLine Text Widget**. The information should be extracted from the properties of the system, as shown in Figure 15.

Note 8: For this application, use the **android.os.Build** object to retrieve the necessary information.

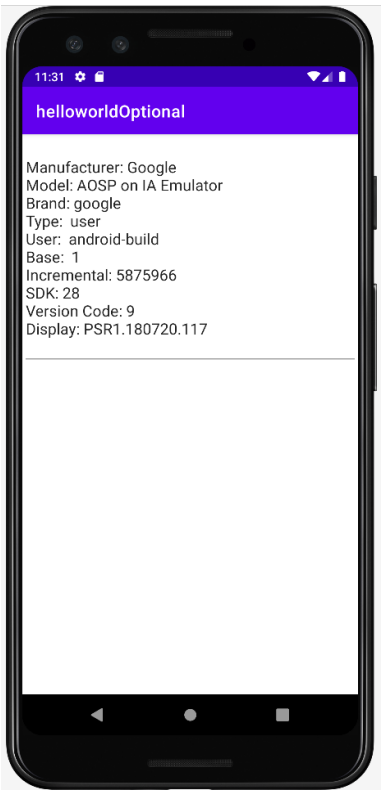

Figure 15: Hello World Optional, showing information about the current build.

6 Exploring Kotlin Again [AC NO, AI NO]

At this point in your assignment, you should have already covered the concepts of **Classes and Objects** in class. Now, let us return to console programming using **IntelliJ IDEA** for a practical exercise. Through this assignment, you will master object-oriented programming concepts in Kotlin, understand inheritance and polymorphism, practice custom getters and setters with business logic, implement collection management, and work with abstract methods and companion objects. Build in `cm.exer_vl` package.

Assignment - A Virtual Library

You have been hired as a software developer for a **Virtual Library Management System**. Your task is to design a system that manages various types of books and library operations.

- Each book should have common attributes such as title, author, publication year, and available copies.
- Different book formats (digital vs. physical) have unique characteristics that differentiate them from one another.
- The library must support core operations such as borrowing, returning, and searching for books.
- Your implementation must adhere to the object-oriented principles in Kotlin.

Apply your knowledge of classes, objects, inheritance, and encapsulation to build a structured and scalable solution that could serve as the foundation for a real-world library management system.

6.1 Base requirements

Implement the following features in Kotlin to create a comprehensive library management system:

Class `Book` (Base Class)

The `Book` class serves as the base class for all book types in the Virtual Library. It includes the following features:

- **Properties:** `title`, `author`, `publicationYear`, and `availableCopies`.
- A **primary constructor** to initialize these properties.
- A **custom getter** for `publicationYear` that returns:
  - `"Classic"` if the publication year is before 1980.
  - `"Modern"` if the publication year is between 1980 and 2010.
  - `"Contemporary"` otherwise.
- A **custom setter** for `availableCopies` that:
  - Prevents negative values.
  - Prints a warning message if someone tries to set copies to 0: `"Warning: Book is now out of stock!"`
- An `init` block that prints a message when a `Book` object is created, including the title and author.

Classes `DigitalBook` and `PhysicalBook` (Subclasses)

The `DigitalBook` and `PhysicalBook` classes inherit from the `Book` base class and introduce additional properties specific to their formats.

- **`DigitalBook` Class:**

  - Includes an additional property: `fileSize` (in MB).
  - Includes an additional property: `format` (e.g., `"PDF"`, `"EPUB"`, `"MOBI"`).
  - Has its own constructor that calls the parent constructor.

- **`PhysicalBook` Class:**
  - Includes an additional property: `weight` (in grams).
  - Includes an additional property: `hasHardcover`, which is set to `true` by default.
  - Has its own constructor that calls the parent constructor.

Class `Library` (Manager)

The `Library` class is responsible for managing a collection of books within the Virtual Library.

- Stores a **list of books**.
- Implements the method `addBook()` to add new books to the collection.
- Implements the method `borrowBook(title: String)` that:
  - Searches for a book by title.
  - Decreases `availableCopies` by 1 if the book is found and copies are available.
  - Prints an appropriate message for success or failure.
- Implements the method `returnBook(title: String)` that:
  - Searches for a book by title.
  - Increases `availableCopies` by 1 if the book is found.
  - Prints a confirmation message.
- Implements the method `showBooks()` to print details of all books in the library.
- Implements the method `searchByAuthor(author: String)` to display all books by a specific author.

6.2 Extended requirements

Extend the functionality of the classes by implementing the following features:

- **Override the `toString()` method** in each class to provide a custom string representation of the objects.
- **Implement an abstract method `getStorageInfo()`** in the `Book` class, which must be defined in each subclass:
  - For `DigitalBook`: Returns storage information (e.g., `"Stored digitally: 5.2 MB, Format: PDF"`).

  - For `PhysicalBook`: Returns physical information (e.g., `"Physical book: 450g, Hardcover: Yes"`).

- Add a **companion object** to the `Library` class that tracks:
  - Total number of books added across all library instances.
  - A method `getTotalBooksCreated()` to retrieve this count.
- Implement a **data class** `LibraryMember` with properties: `name`, `membershipId`, and `borrowedBooks` (list of book titles).

6.3 Expected Implementation

Listing 1: Example Usage

```
fun main() {
    val library = Library("Central Library")

    val digitalBook = DigitalBook(
        "Kotlin in Action",
        "Dmitry Jemerov",
        2017,
        5,
        4.5,
        "PDF"
    )

    val physicalBook = PhysicalBook(
        "Clean Code",
        "Robert C. Martin",
        2008,
        3,
        650,
        true
    )

    val classicBook = PhysicalBook(
        "1984",
        "George Orwell",
        1949,
        2,
        400,
        false
    )

    library.addBook(digitalBook)
    library.addBook(physicalBook)
    library.addBook(classicBook)

    library.showBooks()

    println("\n--- Borrowing Books ---")
    library.borrowBook("Clean Code")
    library.borrowBook("1984")
    library.borrowBook("1984")
    library.borrowBook("1984")  // Should fail - no copies left

    println("\n--- Returning Books ---")
```

```
    library.returnBook("1984")

    println("\n--- Search by Author ---")
    library.searchByAuthor("George Orwell")
}
```

6.4 Expected Console Output

```
Book 'Kotlin in Action' by Dmitry Jemerov has been added to the library.
Book 'Clean Code' by Robert C. Martin has been added to the library.
Book '1984' by George Orwell has been added to the library.

--- Library Catalog ---
Title: Kotlin in Action, Author: Dmitry Jemerov, Era: Contemporary, Available: 5 copies
  Storage: Stored digitally: 4.5 MB, Format: PDF

Title: Clean Code, Author: Robert C. Martin, Era: Modern, Available: 3 copies
  Storage: Physical book: 650g, Hardcover: Yes

Title: 1984, Author: George Orwell, Era: Classic, Available: 2 copies
  Storage: Physical book: 400g, Hardcover: No

--- Borrowing Books ---
Successfully borrowed 'Clean Code'. Copies remaining: 2
Successfully borrowed '1984'. Copies remaining: 1
Successfully borrowed '1984'. Copies remaining: 0
Warning: Book is now out of stock!
Sorry, -- Returning Books ---
Book '1984' returned successfully. Copies available: 1

--- Search by Author ---
Books by George Orwell:
  - 1984 (Classic, 1 copy available)
```

6.5 Assessment Criteria

Your submission will be evaluated on the following criteria:

- **Correct class and object declaration**;
- **Proper use of primary and secondary constructors**;
- **Usage of the `init` block**;
- **Correct implementation of getters and setters with custom logic**;
- **Proper inheritance structure and use of subclasses**;
- **Functionality of the Library class including all required methods**;

- **Code organization and readability**;
- **Error handling for edge cases (e.g., borrowing unavailable books)**; and
- **The use of abstract methods, companion objects, and data classes** .

7 Return and Enhance your Android experience [10%] [AC YES, AI NO]

Congratulations on reaching this section! By successfully completing all the tasks already described, you have secured 60% of the total grade for this assignment. Now, it is up to you to take the initiative and enhance your Android learning experience by proactively adding additional tasks. You will earn more 10% in your grade, with these actions. Here are some ideas for achieve it:

- Explore and learn how to use **the Android Debug Bridge (ADB)** in console mode;
- Implement a **landscape layout** for your Hello World application; and
- Add new features or improvements according to your interests.

The teacher will evaluate your additional work based on its difficulty, originality, and quality. Take this opportunity to challenge yourself, explore new concepts and deepen your understanding of Android development.

8 Assisted code generation - MIP [20%] [AC YES, AI YES]

Automatic code generation and agentic programming represent a major shift in the way software engineering is approached and practiced. Rather than relying solely on manual coding and traditional development workflows, these technologies enable systems to assist developers by generating code, suggesting solutions, and even taking autonomous actions within the development process.

This evolution is transforming software engineering into a more collaborative effort between humans and intelligent agents, improving productivity, accelerating development cycles, and reducing repetitive tasks. As agentic systems become more capable, they are also reshaping how software is designed, tested, and maintained, marking a significant step toward more adaptive and intelligent development environments.

Your Mission (your Mission Impossible Possible (MIP-1)

In this section, the use of AI is mandatory and you can earn the remaining **20%** of your grade. You may use a variety of plugins, tools, and Large Language Models to support your work, including: **Google Antigravity** (`https://antigravity.google/`), Claude

Code (`https://claude.com/product/claude-code`), GitHub automation tools such as co-pilot, or (`https://github.com/copilot`) depending on what best fits your workflow.

The objective of this task is to develop a new **mobile application** in Android Studio and Kotlin. You are free to decide on the type of application and its level of complexity, which will be considered during the evaluation. You are invited to explore other API that work with images. Try this site (or any other): https://apipheny.io/free-api/

For Example. You may build a **weather application** with a map in the main screen. The user should be able to tap on a location on the map and view the corresponding weather information displayed in another screen. This basic concept can then be expanded with additional features such as forecasts, saved locations, or live updates.

You are encouraged to use all available AI tools to support your learning, but you must make all the effort to understand the generated code and how the development environment works. Use the AI to explain the code for you. This includes understanding how the code is deployed to both physical and virtual devices, as well as how debugging is performed. **When you present this task to your teacher, you will be questioned about these aspects**.

AI-Assisted Software Engineering Tools

We recommend using **Google Antigravity**. In Antigravity, you can specify mobile application requirements using two primary methods:

- **Direct prompting**, best suited for active development and quick iteration; and
- **Markdown-based configuration**, useful for persistent project rules and automated workflows.

Direct Prompting (Best for Rapid Start)

The most common way to begin building an app is to provide a detailed prompt in the **Agent Manager**.

- **Planning Mode:** Ensure that the agent is set to *Planning Mode* (not Fast Mode) via the side panel. This forces the agent to generate an Implementation Plan and a structured task list before writing any code.
- **Prompt Structure:** A high-quality prompt should include: *Context + Goal + Constraints + Plan + Verification + Deliverables*. See text box with title: Example Prompt for APP in Android.
- **Iterative Refinement:** Start with the user interface only, then add logic, persistence, and external APIs in subsequent prompts.

Example Prompt for APP in Android (Kotlin + XML Views)

Context: You are an autonomous software engineering agent tasked with creating a production-quality, greenfield native Android mobile application using Kotlin, XML-based Views, Android Studio, and Material Design 3.
Goal: Create a mobile application named "Weather Buddy" that displays a map on the main screen. The user must be able to tap on a location on the map and view the corresponding weather information on a separate screen.
Constraints: Use the latest stable Android SDK, Kotlin, XML layouts (Views system), Material Design 3 components, and follow widely adopted Android best practices and architectural guidelines. Use an appropriate maps SDK and a public weather API to retrieve weather data for the selected location.
Plan: Before executing any commands or generating code, produce a detailed project plan artifact for approval that defines the recommended architecture (e.g., MVVM), folder structure, key dependencies with justification, theming approach, navigation strategy between screens, state management, assumptions, risks, and scalability considerations.
Verification: Ensure the proposal is complete, maintainable, suitable for future expansion, and compatible with modern Android development standards.
Deliverables: Provide the detailed project plan artifact and wait for explicit approval before implementation.

Markdown Files (Best for Persistent Rules)

If you want to enforce consistent standards across your project, Antigravity supports using `.md` (Markdown) files inside your workspace.

- **Rules (`.agent/rules/*.md`):** These define persistent instructions that the agent always follows, such as coding conventions (e.g., PEP 8, Google Java Style Guide) or modular architecture constraints.
- **Workflows (`.agent/workflows/*.md`):** These are reusable saved prompts triggered by `/commands` in the chat (e.g., `/generate-tests`) to automate repetitive development tasks.
- **Skills (`SKILL.md`):** Advanced users can define specialized "Skills" in dedicated folders, allowing the agent to load tools or scripts only when relevant to the current request.

Overall, Antigravity provides an excellent opportunity to explore AI-assisted development workflows in a realistic software engineering environment. Go ahead, and use AI to learn more about this.

Procedure Summary in the Antigravity

1. **Open Workspace:** Create a new workspace (directory).
2. **Set Mode:** Switch the agent conversation mode to *Planning Mode*.

3. **Define Specifications:** Input your mobile app requirements (e.g., use Kotlin and XML Views) via a detailed prompt.
4. **Review Plan:** Approve the artifacts generated by the agent in the implementation plan.
5. **Authorize Execution:** Click *Accept* on proposed terminal commands to allow the agent to modify your file system or execute commands (like checking version version, find in files, replace in files, ...) if the command looks OK to you.
6. **Verify Results:** Use the integrated emulator or browser preview to test the agent's work through generated walkthrough artifacts.
7. **Add version control:** Click on the Source Control icon (below the Code Search icon) to add the code to a local Git repository - press Initialize Repository; then Generate a comment to the current changes; and press Commit (then select Save all changes and Commit option).
8. **Iterate:** Iterate as many times as necessary with the agent until your app satisfies all functional and design requirements.

Have fun and good vibes!